

Algorithme de résolution d'équation, implémentation de Gauss-Jordan

August 10, 2026

1 Introduction

On recherche le vecteur y tel que le produit $M \cdot y = a$

$$\begin{bmatrix} M_{1,1} & \cdots & M_{1,j} \\ \vdots & \ddots & \vdots \\ M_{i,1} & \cdots & M_{i,j} \end{bmatrix} \cdot \begin{bmatrix} y_1 \\ \vdots \\ y_i \end{bmatrix} = \begin{bmatrix} a_1 \\ \vdots \\ a_i \end{bmatrix}$$

L'algorithme de Gauss-Jordan consiste à construire une matrice diagonale en annulant la variables j d'une ligne i à l'aide d'une autre ligne, contenant la variable, nommée pivot.

2 Mon implémentation

2.1 recherche du pivot

Mon implémentation de cet algorithme cherche la première ligne i contenant une variable j non-nulle. Si aucune ligne ne satisfait cette contrainte, l'algorithme passe à la variable suivante. Les lignes utilisées sont échangées avec la m -ième ligne, m étant le nombre de pivots utilisés.

$$\begin{aligned} &\rightarrow \begin{bmatrix} 0 & 2 & 1 \\ \boxed{1} & 0 & 3 \\ 4 & 1 & 1 \end{bmatrix} \begin{matrix} (j = i) \\ (i = 1) \end{matrix} \implies \begin{bmatrix} 1 & 0 & 3 \\ 0 & 2 & 1 \\ 4 & 1 & 1 \end{bmatrix} \end{aligned}$$

La recherche se fait donc sur les $n - m$ lignes restantes avec n , la taille de la matrice M . Si aucune variable n'est libre, alors la recherche des lignes pivots se fait en temps $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ soit $\mathcal{O}(n^2)$.

première variable	deuxième variable	troisième variable
$\rightarrow \begin{bmatrix} 0 & 2 & 1 \\ \boxed{1} & 0 & 3 \\ 4 & 1 & 1 \end{bmatrix}$	$\rightarrow \begin{bmatrix} 1 & 0 & 3 \\ 0 & \boxed{2} & 1 \\ 4 & 1 & 1 \end{bmatrix}$	$\rightarrow \begin{bmatrix} 1 & 0 & 3 \\ 0 & 2 & 1 \\ 4 & 1 & \boxed{1} \end{bmatrix}$

2.2 élimination

Une fois le pivot trouvé, on cherche à éliminer la variable visée dans les autres lignes de la matrice. Si la ligne pivot est la ligne j , alors on élimine la variable visée sur les $n - 1$ lignes $k \in [j + 1; n + j - 1] \bmod n$.

$M_k = M_j \frac{M_{ki}}{M_{ji}}$ et $a_k = a_j \frac{M_{ki}}{M_{ji}}$. comme les précédentes variables ont été annulées, il n'est pas nécessaire d'itérer sur les premières colonnes. le nombre d'itération est donc dans tous les cas $\frac{n(n+1)}{2}$ soit $\mathcal{O}(n^2)$.

L'élimination se fait donc en temps $\frac{n(n+1)(n-1)}{2}$ soit $\mathcal{O}(n^3)$.

2.3 identification

Une fois la matrice diagonale construite, il suffit d'identifier la valeur de chaque variable à l'aide du vecteur a construit précédemment. Comme les m -ièmes pivots ont substitué les lignes m -ièmes lignes de la matrice, $a_i = \frac{a_i}{M_{i,i}}$. Si $M_{i,i}$ est nul, alors, la variable i est dite libre.

Cette identification se fait en n opérations, soit $\mathcal{O}(n)$.

2.4 complexité totale

La complexité totale s'obtient en sommant les complexités de chaque étape. on peut définir une fonction $C(m)$ donnant la complexité en fonction du nombre de pivots m et de la taille n du problème.

fonction de recherche : (Cette formule correspond aux pivots regroupés en début de matrice. Ce n'est pas le cas qui maximise la recherche seule (qui serait obtenu avec les pivots en fin de matrice), mais celui qui maximise le total, comme montré plus bas)

$$C_{\text{recherche}}(m, n) = \frac{m(m+1)}{2} + (n-m)n$$

fonction d'élimination : (la complexité de l'élimination est maximale lorsque le pivot trouvé est situé au début de la matrice)

$$\begin{aligned} C_{\text{élimination}}(m, n) &= (n-1) \left[\sum_{i=1}^n i - \sum_{i=1}^{n-m} i \right] \\ &= (n-1) \left[\frac{n(n+1)}{2} - \frac{(n-m)(n-m+1)}{2} \right] \\ &= (n-1) \frac{m(2n-m+1)}{2} \end{aligned}$$

fonction d'identification :

$$C_{\text{identification}}(m, n) = n$$

On peut remarquer que le cas le plus coûteux en recherche contredit le cas le plus coûteux en élimination. Il s'agit donc de trouver le maximum pour la fonction de recherche et la fonction d'élimination en fonction de l'arrangement des lignes de la matrice afin d'optimiser le résultat.

On introduit une variable Σ représentant la position des pivots : $\Sigma = \sum_{j=\text{pivot}} j$. On cherche à savoir dans quel configuration $C_{\text{recherche}} + C_{\text{élimination}}(\Sigma)$ est maximale.

On a :

$$C_{\text{recherche}}(\Sigma) = n^2 - mn + \Sigma$$

$$C_{\text{élimination}}(\Sigma) = (n-1)(m(n+1) - \Sigma)$$

$$\frac{d}{d\Sigma} C_{\text{recherche}} + \text{élimination} = 2 - n$$

La fonction selon sigma pour $n > 2$ est donc décroissante monotone.

$$\Sigma_{\min} = \sum_{i=1}^m i = \frac{m(m+1)}{2}$$

Soit $C(m, n)$, la fonction de complexité totale de l'algorithme :

$$\begin{aligned} C(m, n) &= C_{\text{recherche}}(m, n) + C_{\text{élimination}}(m, n) + C_{\text{identification}}(m, n) \\ &= \frac{m(m+1)}{2} + (n-m)n + (n-1)\frac{m(2n-m+1)}{2} + n \end{aligned}$$

Dans tous les cas, la complexité totale est $\mathcal{O}(n^3)$.